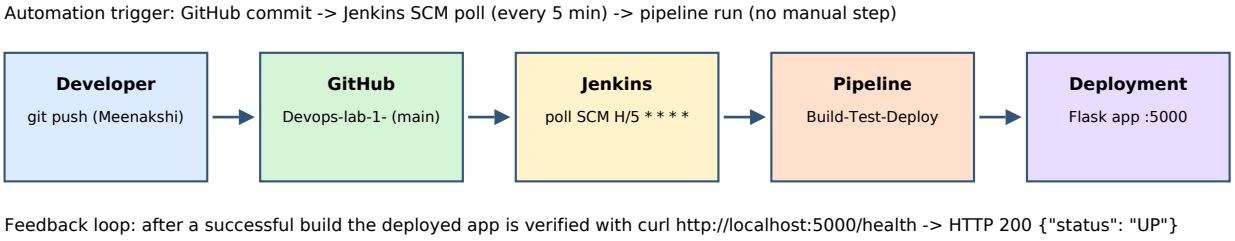
# Question 2: Prepare a CI/CD workflow report describing the pipeline architecture, stages, automation results, and observations with supporting screenshots.

Executed on Ubuntu 24.04 LTS (WSL2) by Meenakshi Rakesh. Repository: https://github.com/meenakshi16rp/Devops-lab-1-

## 1. Pipeline architecture




- Source control: GitHub repository Devops-lab-1- (main branch) owned by meenakshi16rp; the developer pushes commits with a feature-branch workflow.
- Automation trigger: Jenkins polls the repository every 5 minutes (H/5 * * * *). When new commits are found the pipeline starts on its own - no manual build step.
- CI/CD engine: Jenkins 2.580.2 LTS with a declarative pipeline (Jenkinsfile) checked out from the same repository - the pipeline itself is version controlled.
- Target: the built application is deployed to a local deployment directory and served on port 5000; the pipeline verifies the health endpoint before declaring success.

## 2. Pipeline stages

| Stage | What it does | Tool / command | Result |
|---|---|---|---|
| Checkout SCM | Jenkins fetches the Jenkinsfile from main | Git + github-creds | 1 s, OK |
| Checkout | Pulls source code of the latest commit | checkout scm | 1 s, OK |
| Build | Creates a Python virtual environment and installs dependencies | python3 -m venv + pip install -r app/requirements.txt | 8 s, OK |
| Test | Runs the unit test suite | pytest app/tests -v (3 tests) | 0.75 s, OK |
| Deploy | Copies the app to the deploy directory, (re)starts it, verifies health | ./deploy/deploy.sh + curl :5000/health | 2 s, OK |
| Post Actions | Success/failure notification echo; workspace cleanup | post + cleanWs() | 0.38 s, OK |

## 3. Automation results

| Build | Trigger | Commit | Result | Duration |
|---|---|---|---|---|
| #1 | Manual (first run) | eedda81 | FAILURE (Deploy stage) | 30.6 s |
| #2 | Manual (after fix) | c9df47c | SUCCESS | 18.7 s |
| #3 | Automatic - Started by an SCM change | eaa01d3 | SUCCESS | 18.3 s |

- All three unit tests (index, health, info) pass in every successful build.
- The deployed application answers /health with {"status":"UP"} and /info reports author Meenakshi and the Jenkins declarative CI/CD pipeline.

## 4. Evidence

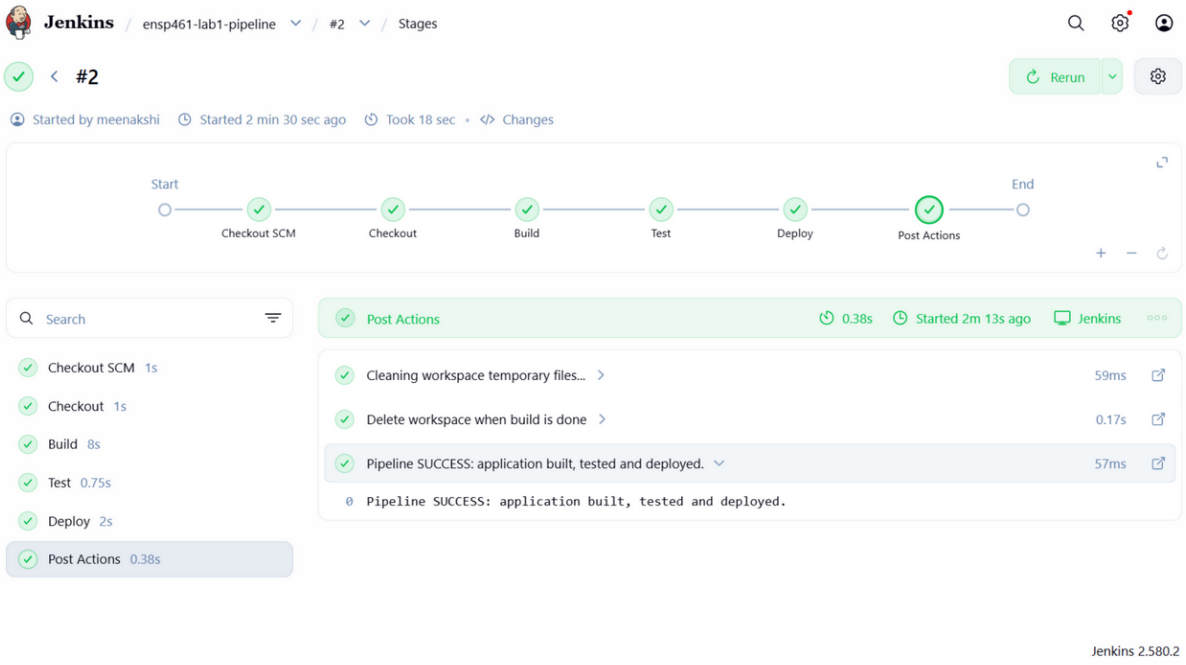



Figure: Stage graph of a complete successful run




Figure: Git polling log: the poll detects the new commit and reports 'Changes found'

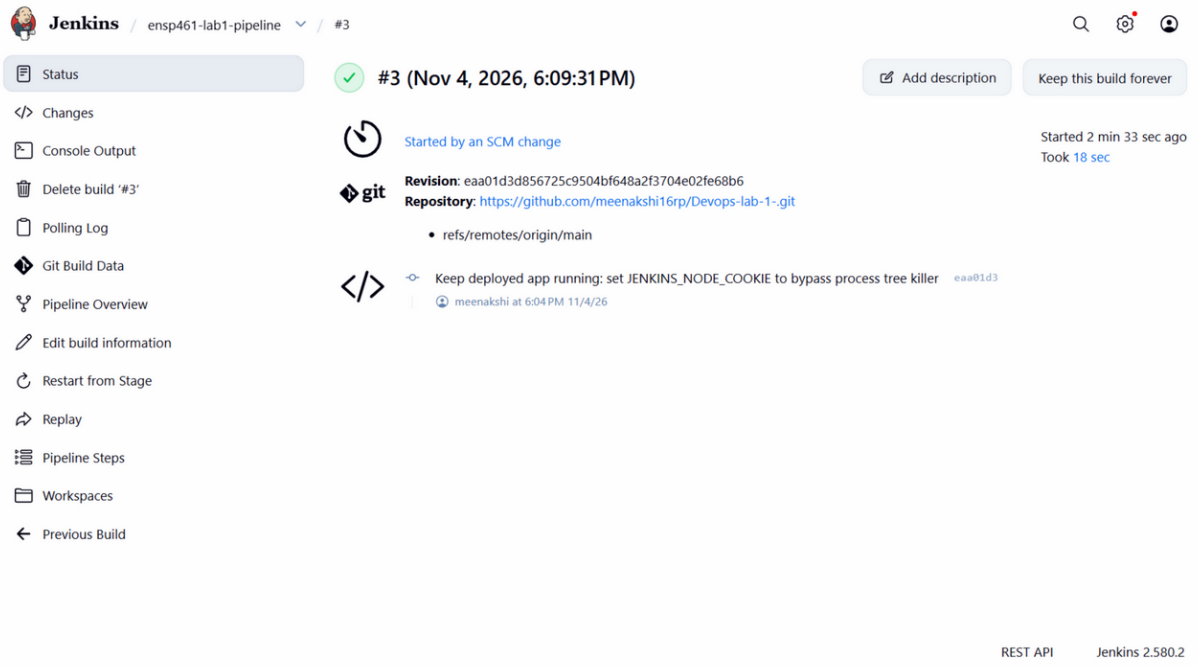



Figure: Build #3 was started automatically by an SCM change

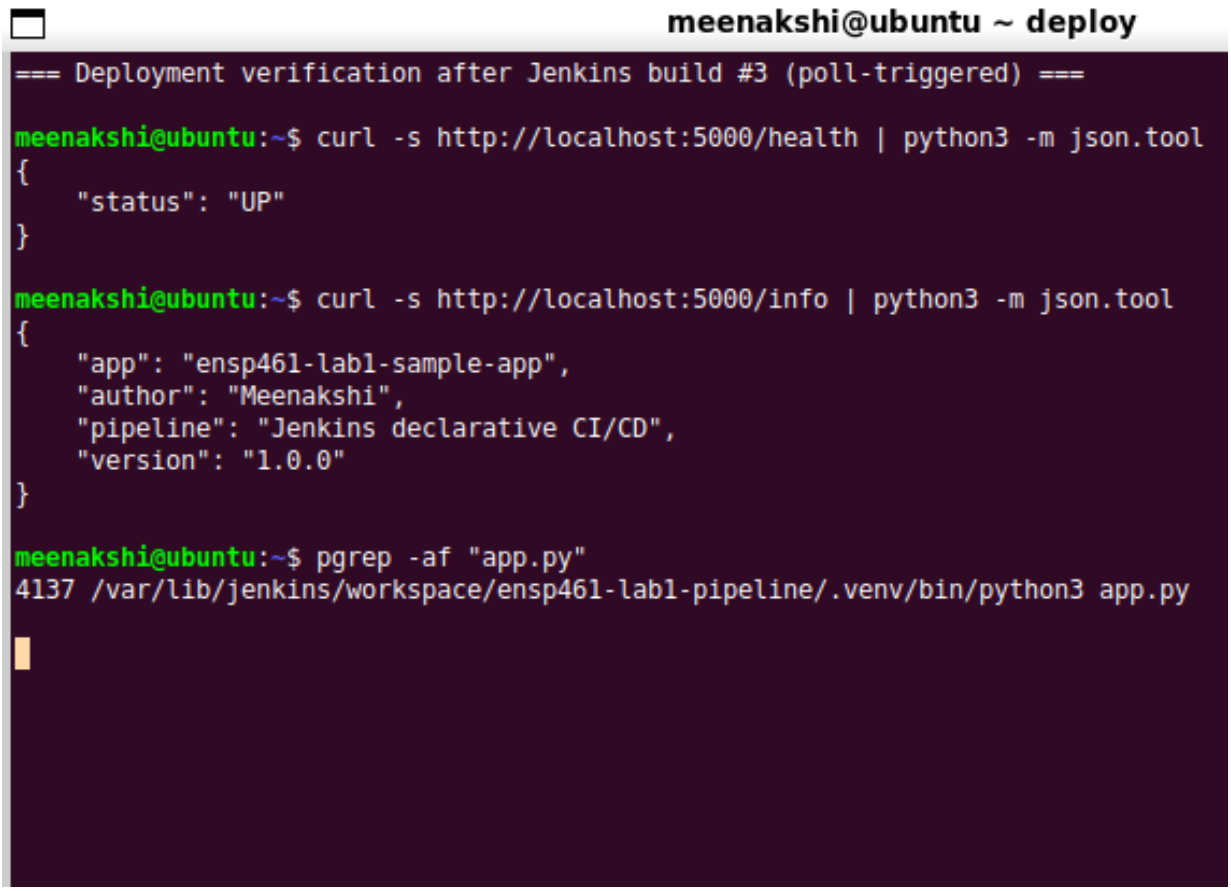



Figure: Deployed application verified live on port 5000

## 5. Observations

- Pipelines-as-code made the whole workflow reproducible: recreating the job only requires pointing Jenkins at the repository.
- The first run failed exactly where an undebugged script typically fails (deployment environment); the console log pinpointed the missing Flask module within seconds - a

good demonstration of why each stage should fail fast.

- Poll SCM is a simple, reliable trigger for environments where GitHub cannot reach the Jenkins host for webhooks (the WSL2 lab is not publicly reachable).
- Cleanup (cleanWs) in the post section keeps the workspace small across builds.